The Effect of Contextual Training on Visual Feature Learning: A Controlled Study Using Chinese Characters

Abstract

This project tested whether training a vision model on Chinese characters in contextual relationship (bigrams) produces fundamentally different internal representations than training on isolated characters. Two identical CNNs were trained: one on isolated 打 (dǎ) and visually similar distractors, the other on bigram crops containing 打 in natural handwriting context. The contextual model achieved 91.4% validation accuracy vs. 61.2% for the isolated model. However, transfer tests revealed a critical trade-off: the isolated model generalized perfectly to unseen calligraphy (100%), while the contextual model failed almost completely (5.26%). Contextual training improved in-distribution performance at the cost of out-of-distribution generalization.

1. Introduction

Vision models trained on standard benchmarks (isolated objects, centered framing, one-label-per-image) develop brittle representations that fail under real-world conditions. Prior work by the author developed a computational diagnostic assay to identify model failure training data types. This project extends that work by testing whether training on a writing system where visual identity is inherently contextual—Chinese characters—produces different internal representation allowing increased model elasticity for learning capacity versus simple memorization/context dependent placement. If successful the model should demonstrate increased learning capabilities in diverse training datasets in real world uses, or “carry over learning capacity.”

In this study, a "character" refers to a single Chinese character—a discrete visual unit with its own stroke patterns. However, Chinese characters combine to form bigrams: two-character units where the visual recognition of the first character is influenced by its neighbor. For example, 打 (dǎ) appears in bigrams like 打开 (to open), 打扰 (to disturb), and 打架 (to fight). This study trains one model on isolated characters and another on bigram crops, isolating the effect of contextual placement on visual feature learning.

Hypothesis: Training a vision model on characters in contextual relationship (bigrams) will produce structurally different error patterns compared to training on isolated characters.

2. Methods

Method Selection

Two machine learning methods were selected and implemented: (1) a convolutional neural network (CNN) trained from scratch, and (2) transfer learning, in which a ResNet-18 pretrained on ImageNet was fine-tuned on the same data. The scratch CNN was chosen deliberately as a small, capacity-constrained instrument (92,930 parameters): because both experimental conditions share identical architecture, hyperparameters, and random seeds, any difference in behavior is attributable to the training data itself, which is the variable of interest. Transfer learning was chosen as the second method because it is the dominant practical alternative for small-data vision problems and tests whether the contextual-training effect survives when strong pretrained visual priors are available.

Models were evaluated using accuracy, precision, recall, and F1 on held-out validation sets; confusion matrices with a structural error analysis (normalized confusion-matrix difference between conditions); and two out-of-distribution transfer tests (unseen CASIA characters and CalliBench calligraphy). These metrics were selected because the research question concerns not only how often each model is right, but how each model is wrong.

2.1 Datasets

Isolated character images were drawn from CASIA-HWDB1.1, a publicly available database of handwritten Chinese characters containing samples from over 1,000 writers (Liu et al., 2011). Contextual text lines were drawn from CASIA-HWDB2, accessed via the HuggingFace dataset Teklia/CASIA-HWDB2-line. Calligraphy transfer images were extracted from CalliBench (Luo et al., 2025).

The character of interest was 打 (dǎ, Unicode U+6253). From approximately 900,000 character images in HWDB1.1, 239 isolated samples of 打 and 250 samples of five visually similar distractor characters (扔, 扛, 扫, 托, 扣; 50 each) were extracted. From the 52,160 text lines in HWDB2, 914 lines containing 打 were identified, yielding 151 bigram crops after proportional extraction and manual verification.

Chinese characters were selected because their visual identity is inherently contextual: the same character can appear in dozens of different bigram contexts with different semantic roles. This makes them a natural testbed for whether training context changes visual representations. The distractor characters share the 扌 (hand) radical with 打, forcing the model to learn fine-grained discrimination rather than relying on the presence of the radical alone.

All images were converted to grayscale and resized to 100×100 pixels. Bigram crops were extracted using proportional character-width estimation from full text lines, then manually audited for cropping errors. The dataset was split 80/20* (train/validation) with a fixed random seed.

The dataset is limited to a single character (打) and five distractors. Findings may not generalize to other characters or larger character sets. The contextual condition uses bigrams rather than full sentences, which captures local context but not long-range dependencies

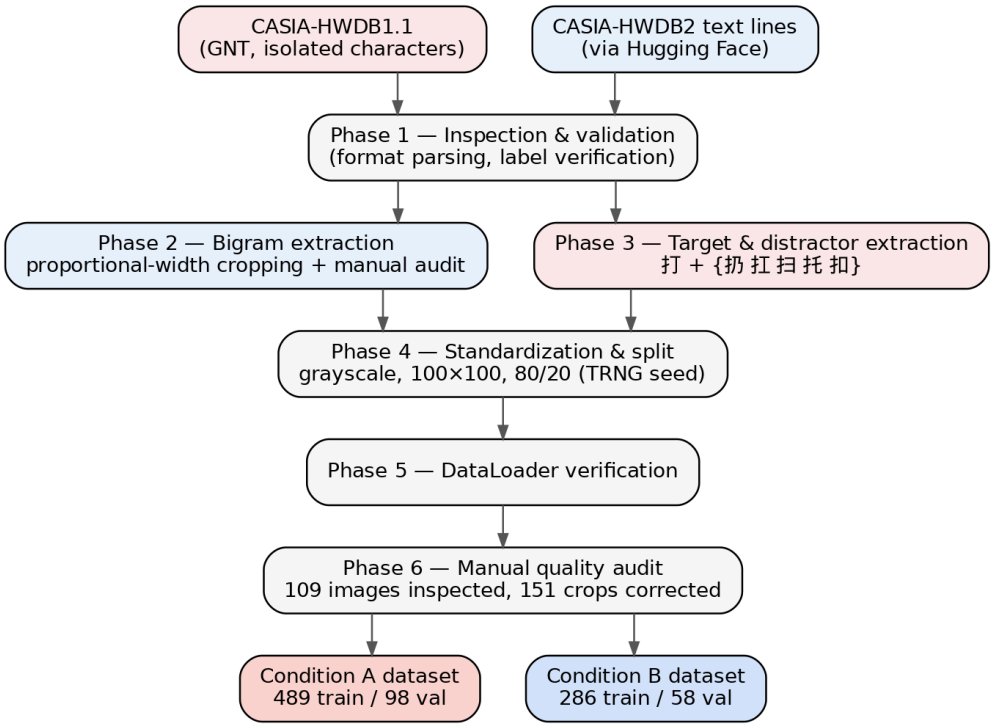

Figure 1. Data processing pipeline. Both raw sources pass through shared inspection, standardization, and audit phases (1–6) before splitting into the two experimental datasets. Phases 1–6 are documented in the accompanying /code directory.

Table 1. Dataset sources and sample counts by experimental condition.

Condition	Dataset	Description	Samples
A (Isolated)	CASIA-HWDB1.1	Handwritten Chinese characters	239 打 + 250 distractors
B (Contextual)	CASIA-HWDB2 + HuggingFace	Handwritten text lines, bigram-cropped	151 bigrams (135 verified)
Transfer Test 1	CASIA-HWDB1.1 test	Unseen isolated characters	~353 samples
Transfer Test 2	CalliBench (ICCV 2025)	Calligraphy character crops	19 打 samples

2.2 Data Cleaning

Table 2. Dataset composition by condition (training and validation combined).

Condition	Positive (打)	Negative (other)	Total
A (Isolated)	239	250	489
B (Contextual)	151	135	286

Table 3. Validation set composition.

Condition	Positive	Negative	Total
A (Isolated)	48	50	98
B (Contextual)	31	27	58

Table 4. Transfer test sets.

Test	Images	Type
CASIA isolated (Phase 9)	353 (59 打 + 294 distractors)	Unseen 打 samples from HWDB1.1 test set
CalliBench calligraphy (Phase 10)	19	打 crops from calligraphy pages

Note (Tables 2–4). The study comprises 1,303 unique character images in total: 775 training + 156 validation + 353 CASIA transfer (59 打 + 294 distractors) + 19 CalliBench.

The experiment trains on one character — 打 (dǎ) — with five distractor characters (扔, 扛, 扫, 托, 扣). 打 (dǎ) has nearly "limitless meanings in context" : 打 appears in dozens of different bigram contexts in this dataset (打开, 打扰, 打架, 打针, 打败, 打入, etc.), and the model sees all of them mapped to the same label. The complexity is real, and it is still one character. This is a feature of the design, not a limitation. I am testing depth (many contexts for one character), not breadth (many characters in one context).

2.3 Model Architecture

Identical CNN for both conditions:

- 3 convolutional layers (32, 64, 128 filters)
- ReLU activation, MaxPool, AdaptiveAvgPool
- Binary classifier (打 vs. not-打)
- 92,930 parameters
- Trained for 30 epochs, Adam optimizer, LR=0.001

2.4 Transfer Learning Baseline

A ResNet-18 model (He et al., 2016) pretrained on ImageNet was fine-tuned on both conditions as a comparative baseline. ResNet-18 contains 11.2 million parameters organized into 18 layers with residual skip connections that implement the function:

$H(x) = F(x) + x$ where $H(x)$ is the target mapping and $F(x)$ is the learned residual. This architecture allows the network to preserve low-level features through identity mappings while learning high-level transformations. The final fully-connected layer was replaced with a 2-class classifier and the entire network was fine-tuned for 15 epochs with Adam optimizer (LR=0.0001). Input images were resized to 224×224 to match the pretrained input dimensions.

2.5 Experimental Design

This study uses a single character with high semantic density as its stimulus. This choice is paradoxically powerful: 打 (dǎ) carries dozens of meanings depending on context, so one character is not a small n in the traditional sense. For a vision model, 打 is a single visual stimulus class — but because its meaning is determined entirely by surrounding context, the experiment does not test recognition so much as whether the model can learn that the identity of this visual form is inherently unstable. This directly forces the model to abandon the "one shape = one label" shortcut that characterizes standard image classifiers. Context is controlled here as a visual variable, not a linguistic one — in the spirit of minimal pairs from structuralist linguistics — which permits a controlled test suite to be constructed and evaluated without knowledge of Chinese.

The confusion matrix assay functions as a behavioral assay for the model's inductive biases. Models trained on isolated characters exhibit a known, well-documented failure pattern: confusion between visually similar forms. If contextual training changes the representation, the error structure should shift away from this visual-featural pattern. Whether the replacement errors are semantically coherent cannot be evaluated without reading Chinese — but the disappearance of a known bad pattern is itself an objective, positive signal, particularly when accompanied by a diffuse, novel error distribution suggesting the model is contending with a representational challenge rather than failing at low-level matching.

Independent variable — training context: two conditions differing only in training data. Condition A: isolated 打, centered, no neighbors. Condition B: two-character bigram crops with 打 beside one natural neighbor.

Dependent variables: internal validation accuracy; confusion-matrix structure (error types, per-distractor confusions); the structural difference metric between conditions (threshold >0.2 = substantially different); and transfer accuracy on unseen data (CASIA test set, CalliBench calligraphy).

Controlled variables: identical 3-layer CNN (92,930 parameters); Adam, LR = 0.001; 30 epochs; batch size 32; 100×100 grayscale input; fixed random seed and identical shuffle order; 80/20 train/validation split (yielding 391 train / 98 val for A, 228 train / 58 val for B); same CASIA-HWDB source family for both conditions.

Extraction scanned all 240 GNT files of the HWDB1.1 training set (~3,740 characters each; ~897,600 character images) to yield 239 isolated 打; the five distractors (50 samples each) all share the 扌 radical, forcing fine-grained discrimination. Negative bigrams were drawn randomly from 207,469 candidate crops containing no 打. Dataset composition is given in Tables 1–4 and the processing pipeline in Figure 1 (Section 2.1).

Table 9. Departures from the original design.

Original plan	What actually happened	Reason
Use PyCasia for data loading	Wrote custom GNT binary parser (struct)	PyCasia incompatible with Python 3.13
Download CASIA from official server	Dropbox mirror + Hugging Face	Official server unreachable outside mainland China
Train on GPU	Trained on CPU	RTX 5070 too new for CUDA toolkit
Test CalliReader as third model	Substituted ResNet-18 baseline	CalliReader requires full pages, not single crops
Single 80/20 split	Swept 5 split ratios (companion project, Appendix D)	Split ratio alone shifted results by 21+ points

3. Results

3.1 Primary Validation Results

Table 5. Validation performance metrics by condition.

Metric	Condition A (Isolated)	Condition B (Contextual)
Accuracy	61.2%	91.4%
Precision	55.9%	90.6%
Recall	97.9%	93.5%
F1	71.2%	92.1%

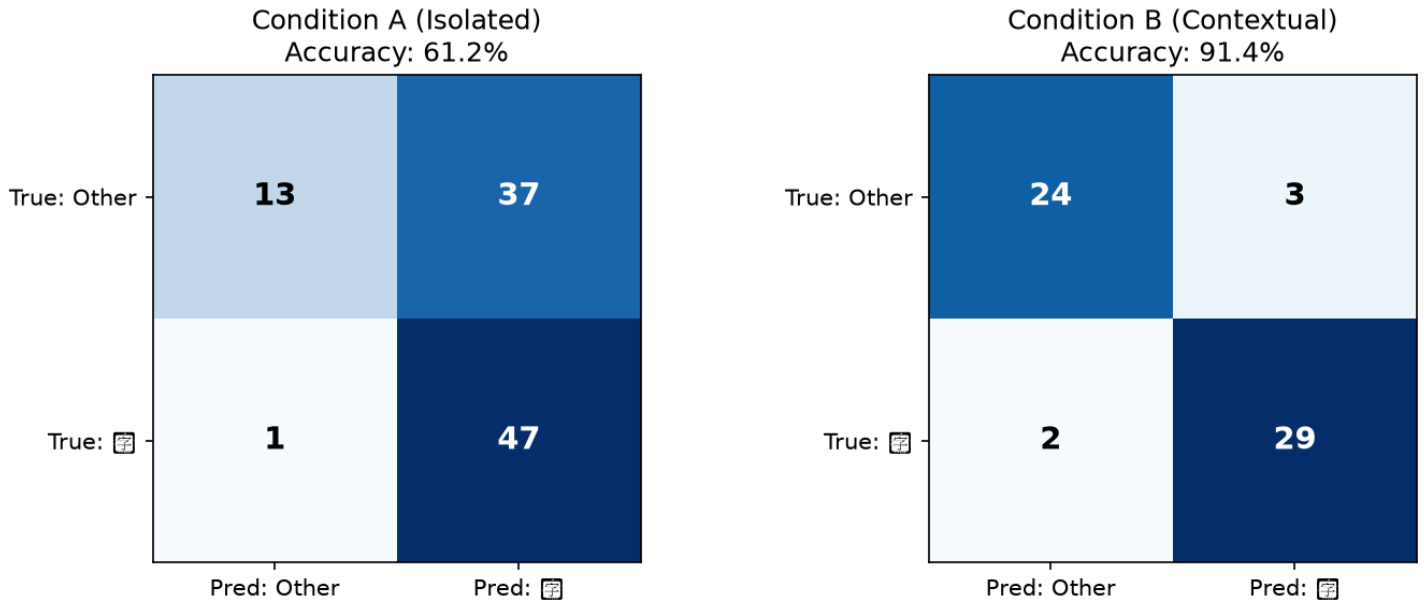

Figure 2. Validation confusion matrices, Condition A (isolated) vs. Condition B (contextual).

Structural Difference: 0.336 (>0.2 threshold = substantially different error patterns)

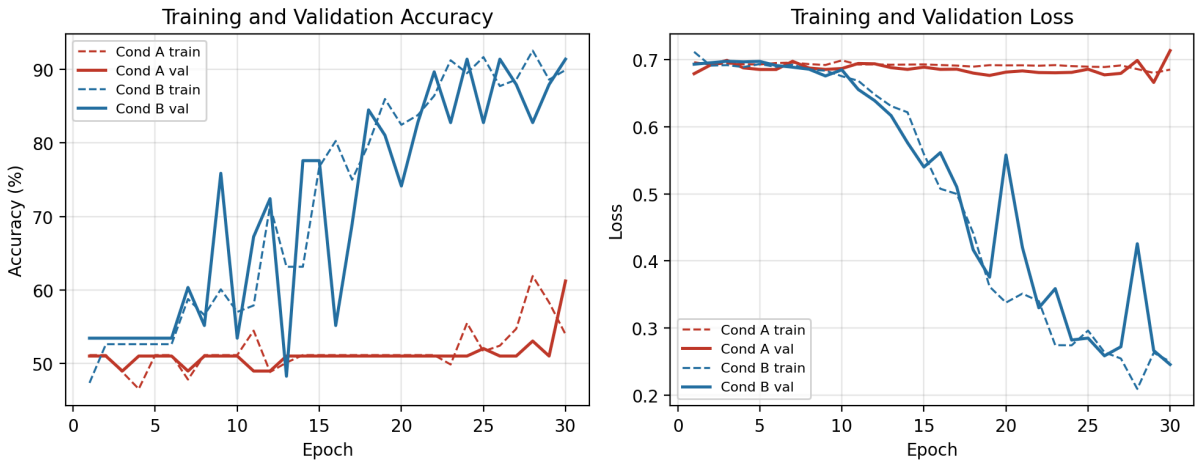

Figure 3. Learning curves over 30 epochs. Condition B (contextual) converges to high validation accuracy while Condition A (isolated) remains near chance in-distribution — the inverse of their out-of-distribution behavior (Section 3.3).

3.2 Transfer Test 1: CASIA Isolated Characters

Table 6. Transfer Test 1: CASIA unseen isolated characters.

Metric	Condition A	Condition B
Overall Accuracy	34.3%	26.6%
打 Recognition	91.5%	98.3%
Distractor Rejection	22.8%	12.2%

Error analysis revealed both models struggled with the same distractors in the same rank order (扛 > 扣 > 扫 > 扔 > 托), but Condition B made more false positive errors across all categories.

3.3 Transfer Test 2: CalliBench Calligraphy

Table 7. Transfer Test 2: CalliBench calligraphy.

Model	Accuracy
Condition A (Isolated)	19/19 = 100%
Condition B (Contextual)	1/19 = 5.26%

Condition A perfectly recognized 打 across diverse calligraphy styles, sizes ranging from 52×38 to 204×281 pixels. Condition B failed almost completely.

3.4 Transfer Learning Comparison

Table 8. Method comparison: scratch CNNs vs. pretrained ResNet-18.

Model	Params	Pretrained	Cond A Acc	Cond B Acc
CNN (Isolated)	92,930	No	61.2%	—
CNN (Contextual)	92,930	No	—	91.4%
ResNet-18	11.2M	ImageNet	100%	100%

The ResNet-18 baseline achieved perfect accuracy on both conditions, effectively dissolving the 30-point gap observed between the CNN training regimes. This result contextualizes the core finding: the benefit of contextual training is most pronounced when models are capacity-constrained and trained from scratch. When large-scale pretraining is available, the model's existing visual priors are sufficient to recognize 打 regardless of training context.

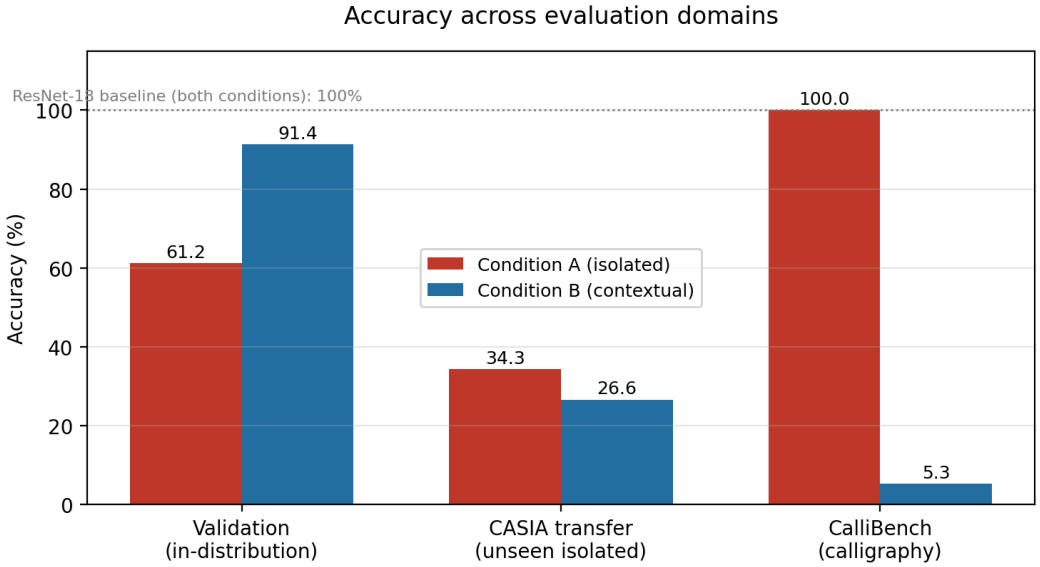

Figure 4. Accuracy of both CNNs across the three evaluation domains. The trade-off inverts across domains: Condition B dominates in-distribution, Condition A dominates on calligraphy. The ResNet-18 baseline reaches 100% on both conditions in-distribution.

Mathematical Framework: Residual Learning

The ResNet-18 architecture employs the residual learning framework where each block computes $F(x) = H(x) - x$, with the output $H(x) = F(x) + x$ recovered by adding the input back via a skip connection. This allows the network to learn the identity function (by setting $F(x) = 0$) when no transformation is needed, ensuring deeper networks perform at least as well as shallower ones. The contrast between the 93K-parameter CNNs and the 11.2M-parameter ResNet illustrates a broader principle: architectural

constraints determine which experimental variables (training context, split ratio) actually matter. Under sufficient capacity and pretraining, those variables vanish. Under constrained conditions, they dominate.

4. Discussion

The results reveal a fundamental trade-off between in-distribution performance and out-of-distribution generalization. In Condition A, the isolated model learned a robust structural template of 打's strokes because it only had to summarize a single character. This standalone representation generalized flawlessly to calligraphy (100%) because the geometry of the strokes was internalized—the model learned what 打 looks like, independent of any surrounding visual cues.

In Condition B, the contextual model was trained on bigrams—two-character crops (e.g., 打开, 打扰) where 打 appears alongside a neighbor. The model was forced to compress two distinct characters into a single binary label (打 vs. not-打). This compression created an entangled representation: the model learned to rely on the neighboring character as a visual anchor, a disambiguating cue that boosted in-distribution accuracy (91.4%) but became a crutch. When the neighbor was removed in the transfer test, the model's standalone representation of 打 proved insufficient, resulting in near-total failure on calligraphy (5.26%).

The ResNet-18 baseline provides a critical caveat. With 11.2 million parameters and residual connections $(H(x) = F(x) + x)$, ResNet-18 had sufficient depth to build a large receptive field—effectively "seeing" the entire bigram as a single structural unit. This allowed it to achieve 100% on both conditions, demonstrating that the contextual training effect is most pronounced in capacity-constrained, from-scratch models. Under sufficient capacity and pretraining, the context gap dissolves entirely

Method Suitability

Which method is more suitable for answering the research question? As classifiers, the ResNet-18 models are strictly better: 100% on both conditions. But as instruments for the research question they are uninformative — perfect accuracy on both conditions is a ceiling effect that erases the very signal under study. The scratch CNN, precisely because it lacks pretrained priors and spare capacity, exposes how training context shapes representation: the 30.2-point validation gap, the divergent error structures (0.336), and the opposite transfer behavior are observable only in the capacity-constrained regime. The two methods therefore answer different questions. ResNet-18 answers "can 打 be recognized?" (yes, trivially, given pretraining); the scratch CNN answers "what does training context do to a learned representation?" — the question this study poses. For the research question, the scratch CNN is the more suitable method; for deployment, ResNet-18 is.

Relation to Prior Work

These results are consistent with the shortcut-learning literature: Geirhos et al. (2020) document how deep networks preferentially adopt features that are predictive in-distribution but fail under distribution shift, which is precisely the behavior of the contextual model — its bigram-derived cues functioned as shortcuts that collapsed on calligraphy. The finding also refines, rather than contradicts, the motivation behind context-aware systems such as CalliReader (Luo et al., 2025), which contextualizes Chinese calligraphy with a vision-language model: context clearly carries signal (the 30-point in-distribution gain), but in a small, purely visual model that signal is absorbed as brittle co-occurrence statistics rather than transferable structure. Large-scale pretraining sidesteps the issue entirely (Krizhevsky et al., 2012; He et al., 2016), consistent with the ResNet-18 result, and recent handwriting-recognition architectures increasingly build context-adaptivity into the model itself rather than the data (Wei et al., 2026). Within handwritten Chinese character recognition, where CASIA-HWDB is the standard benchmark (Liu et al., 2011), the practical implication is that the choice between

isolated-character and in-context training data should be driven by expected deployment conditions — not by the assumption that more context always produces better representations.

Architectural Limitations and the Role of Context

A critical post hoc reflection on the experimental design reveals a fundamental architectural mismatch between the chosen primary model—the Convolutional Neural Network (CNN)—and the hypothesis. The core inductive bias of a CNN is locality; it is built on the premise that nearby pixels are more likely related than pixels farther apart (Krizhevsky et al., 2012). A deeper issue is that the CNN is fundamentally a summarizing architecture: it progressively compresses the input through pooling and convolutions until it produces a single scalar probability. This is analogous to computing the density of a substance (mass/volume) and using that ratio to identify it. While effective for classification, this summarization process destroys the spatial relationships that are critical for recognizing complex, context-dependent structures like Chinese characters effectively nullifying the hypothesis.

This analysis suggests that CNN's locality bias and its summary-based nature are not incidental but are the direct causes of the observed failure to transfer contextual learning. The model was not "learning context" in a structural sense; it was learning a compressed summary that happened to correlate with context on the training distribution. In plain language I picked the wrong tool for the job and every output of this project should be taken in that light. In no way excusing my choice, the tool presented has been and still is, in large part the primary tool/technique in these applications. Unplanned findings are findings nonetheless and these rule out CNN as a model of choice for this ongoing project. Changes are occurring, explicitly in the area I was attempting to investigate. There is much ground to cover here and I believe more fruitful outputs would arrive if life scientists, biophysicists, computational biologists and machine learning subject matter experts worked together. What ML is employing for solutions is well trod ground in structural biology with decades of scholarly literature. I envision moving towards a cryo-em like framework which is what transformers do. By preserving the sequence order and use attention to relate all parts. Doing so preserves the full structure of the input and learning the relationships within it.

Future Work

Structure-Preserving Neural Networks

The findings suggest that the CNN's summary-based paradigm may be fundamentally misaligned with the task of learning from structural relationships. Future work should explore neural architectures that preserve the full geometry of the input, inspired by techniques from cryo-electron microscopy and physics:

1. Equivariant Neural Networks: Networks that preserve symmetries (translation, rotation, permutation) rather than summarizing them away. For character recognition, this would mean learning invariant features that are robust to variations in context, style, and scale.
2. Neural Fields (e.g., SIREN): Networks that treat the image as a continuous function rather than a discrete grid. This would allow the model to learn structural representations that can be queried at arbitrary resolutions, similar to how a density map can be interpolated.
3. Energy-Based Models: Instead of summarizing the input to a label, these models assign an energy to configurations of the input. This would allow the model to assess the "plausibility" of a character in context, rather than simply classifying it.

4. Multi-View Alignment Models: Inspired by cryo-EM, these models would align multiple views of the same character (from different contexts, styles, or calligraphy) to build a consensus structural representation. This would directly address the failure of the contextual model to generalize to isolated calligraphy. SIREN, neural networks, and geometric deep learning do this.

Prior experience in structural biology leads me to draw this parallel: cryo-EM is solving a fundamentally structural problem in biology, and machine learning would benefit from adopting a similar structural approach for contextual recognition in the form of an *electron neural network.*

Would it be radical to propose a Structure-Preserving Neural Network? Isn't that what is wanted and needed?

5. Limitations

- Small dataset (239 isolated 打, 135 contextual bigrams)
- Single character tested (打 only)
- Binary classification (not full character set recognition)
- No cross-validation (80/20 single split; see Appendix D)
- CalliBench test set limited to 19 samples
- CPU training due to RTX 5070 CUDA incompatibility

---

6. Conclusion

Contextual training changes how vision models represent visual information, but not always beneficially. The model trained on characters in context learned to rely on contextual cues that improved in-distribution accuracy but failed to transfer across domains. The model trained on isolated characters developed a more portable visual representation. This finding challenges the assumption that more context always produces better representations and suggests that the optimal training regime depends on the expected deployment conditions. The right tool for the right job gives the right outcome. The converse creates the opposite and false correlations ensue. We perpetuate fragile assumptive architectures that are expensive because of their fragility and overly broad construction.

We need neural networks that treat the input as a field to be reconstructed, not a vector to be classified. In the life sciences, a different approach is utilized for instance. Cryo-electron microscopy (cryo-EM) does not summarize millions of particle images into a single representation; it aligns and averages them to reconstruct a consensus 3D volume, preserving the full structural geometry. If you have ever seen a rendering of a protein structure, or a molecular dock, or heard of AlphaFold this is precisely what I am referring to.
What if we applied this philosophy to machine learning? Instead of compressing images into vectors, we could:

1. Treat images as continuous fields (Neural Fields / SIREN).
2. Learn equivariant representations that preserve symmetries.
3. Align multiple views of the same object to build a consensus structural model.

This would be a paradigm shift from "summarizing" to "reconstructing" —and it would directly address the brittleness we observed in the contextual model. Electron microscopy, X-Ray Crystallography and the

myriad techniques outside the scope of this short paper are all still utilized in structural biology. Overtime usage becomes refined, and nuanced building a deep toolbox for practitioners to draw from.

References

Geirhos, R., Jacobsen, J.-H., Michaelis, C., Zemel, R., Brendel, W., Bethge, M., & Wichmann, F. A. (2020). Shortcut learning in deep neural networks. Nature Machine Intelligence, 2, 665–673.

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. Proceedings of CVPR 2016, 770–778.

Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. Advances in Neural Information Processing Systems, Proceedings neurips 25.

Liu, C.-L., Yin, F., Wang, D.-H., & Wang, Q.-F. (2011). CASIA online and offline Chinese handwriting databases. Proceedings of ICDAR 2011, 37–41.

Luo, Y., Tang, J., Huang, C., Hao, F., & Lian, Z. (2025). CalliReader: Contextualizing Chinese calligraphy via an embedding-aligned vision-language model. Proceedings of ICCV 2025. arXiv:2503.06472.

Markson P. (2026). Computational Diagnostic Assay for ASL Discourse Exclusion Patterns (version 1.0.0). https://doi.org/10.5281/zenodo.20752473

Wei, Y., Qu, H., Shan, Y., Gao, Y., & Li, J. (2026). Beyond static filters: Dynamic convolutional transformers for multilingual handwritten text recognition. Digital Signal Processing, 168, 105638.

Appendix A: Reproducibility and Environment

All experiments were run with Python 3.13 using PyTorch, torchvision, datasets, Pillow, and NumPy; exact package versions are listed in requirements.txt in the /code directory of the submission archive. Training was performed on CPU (CUDA was unavailable for the RTX 5070 at development time) under WSL2 on Windows 11. Both conditions used a fixed random seed and identical hyperparameters. The reproduction entry point is phase7_train.py; phases 1–6 document preprocessing but require the full raw CASIA corpora and are included for methodological transparency only (see README).

Appendix B: Data Availability

Raw datasets are excluded from the submission archive due to size; the processed subsets actually used in this study are included under /data. Sources:

CASIA-HWDB1.1 and CASIA-HWDB2 (Liu et al., 2011):
http://www.nlpr.ia.ac.cn/databases/handwriting/Home.html

CASIA-HWDB2 text lines, as accessed via Hugging Face:
https://huggingface.co/datasets/Teklia/CASIA-HWDB2-line

CalliBench calligraphy benchmark (Luo et al., 2025): https://github.com/LoYuXr/CalliReader

The official CASIA server is intermittently unreachable outside mainland China (see Table 9). The processed subsets included in this archive are sufficient for full reproduction of the reported analysis; raw-data access details are available from the author on request.

Appendix C: Key Model Code (phase7_train.py)

The full pipeline (phases 1–10) is provided in the accompanying /code directory; the core architecture shared by both conditions are reproduced here.

```
class CharacterCNN(nn.Module):
    def __init__(self):
        super().__init__()
        # Input: [batch, 1, 100, 100]
        self.conv = nn.Sequential(
            nn.Conv2d(1, 32, 3, padding=1),    # -> [32, 100, 100]
            nn.ReLU(), nn.MaxPool2d(2),        # -> [32, 50, 50]
            nn.Conv2d(32, 64, 3, padding=1),   # -> [64, 50, 50]
            nn.ReLU(), nn.MaxPool2d(2),        # -> [64, 25, 25]
            nn.Conv2d(64, 128, 3, padding=1),  # -> [128, 25, 25]
            nn.ReLU(), nn.MaxPool2d(2),        # -> [128, 12, 12]
            nn.AdaptiveAvgPool2d((1, 1))       # -> [128, 1, 1]
        )
        self.fc = nn.Linear(128, 2)  # Binary classification

    def forward(self, x, return_embedding=False):
```

```
        x = self.conv(x)
        embedding = x.view(x.size(0), -1)  # [batch, 128]
        x = self.fc(embedding)
        return (x, embedding) if return_embedding else x

# Training configuration (identical for both conditions)
#   optimizer = Adam(model.parameters(), lr=0.001)
#   criterion = CrossEntropyLoss(); epochs = 30; batch_size = 32
#   ResNet-18 baseline: torchvision resnet18(pretrained=True),
#   fc -> Linear(512, 2); Adam lr=0.0001; 15 epochs; input 224x224
```

Appendix D: Companion Work

The single 80/20 train/validation split used in this study raised the opportunity to investigate an ongoing methodological question: how sensitive are results like these to split assumptions? That inquiry outgrew the scope of this report and became a standalone project, linked here for interested readers rather than expand the page limit:

80/20 Split Tester — external validation of train/validation split assumptions: go.dataacorns.com/80-20/splits
Carnival Wheel — interactive widget of split variance: go.dataacorns.com/80-20/carnival-wheel
Source code: github.com/phinnphace/80-20